Guia Rápido de Scikit-Learn — Mineração de Dados

Revisão e Consulta Prática para Prova

1. Fluxo Básico e Divisão dos Dados

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Leitura dos dados
df = pd.read_csv("arquivo.csv")
X = df.drop(columns=["target"])
y = df["target"]

# Divisão Treino e Teste (stratify mantém a proporção das classes)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

---

2. Pré-Processamento e Transformação

Escalonamento Numérico

> **Atenção:** Aplique `fit_transform` **apenas** no treino e `transform` no teste para evitar vazamento de dados (*data leakage*).

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

# StandardScaler: média = 0, desvio padrão = 1 (ideal para SVM, KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# MinMaxScaler: normaliza para o intervalo [0, 1]
minmax = MinMaxScaler(feature_range=(0, 1))

# RobustScaler: usa mediana e IQR (robusto a outliers)
robust = RobustScaler()
```

Codificação de Categóricos

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, LabelEncoder

# LabelEncoder: apenas para a variável ALVO (y)
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

# OneHotEncoder: para variáveis nominais (sem ordem)
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
X_train_ohe = ohe.fit_transform(X_train[['categoria']])
```

```
# OrdinalEncoder: para variáveis ordinais (com hierarquia)
categories_order = [['Fundamental', 'Médio', 'Superior']]
oe = OrdinalEncoder(categories=categories_order)
X_train_ord = oe.fit_transform(X_train[['grau_instrucao']])
```

Tratamento de Valores Ausentes (Imputação) e Binning

```
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import KBinsDiscretizer

# Imputação pela média, mediana ou moda (most_frequent)
imputer = SimpleImputer(strategy='median')  # 'mean', 'median', 'most_frequent', 'constant'
X_train_imp = imputer.fit_transform(X_train)

# Imputação baseada nos k vizinhos
knn_imp = KNNImputer(n_neighbors=5)
X_train_knn_imp = knn_imp.fit_transform(X_train)

# Discretização (Binning)
binner = KBinsDiscretizer(n_bins=5, encode='ordinal', strategy='uniform') # 'quantile', 'kmeans'
X_binned = binner.fit_transform(X_train[['idade']])
```

ColumnTransformer e Pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

num_features = ['idade', 'salario']
cat_features = ['escolaridade', 'cargo']

preprocessor = ColumnTransformer(transformers=[
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')), ('scaler', StandardScaler())]), num_features
    ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')), ('ohe', OneHotEncoder(handle_unknown=
])
```

---

3. Modelos de Classificação e Hiperparâmetros

3.1. K-Nearest Neighbors (KNN)

Dica: Sempre normalize os dados antes de rodar o KNN!

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5,          # Valor de k (ímpar reduz empates)
    weights='uniform',      # 'uniform' (todos iguais) ou 'distance' (mais próximos pesam mais)
    metric='minkowski',     # 'euclidean' (p=2), 'manhattan' (p=1), 'minkowski'
    p=2                     # p=1: Manhattan | p=2: Euclidiana
)
knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)
```

3.2. Árvores de Decisão (Decision Tree)

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

dt = DecisionTreeClassifier(
```

```
    criterion='gini',          # 'gini', 'entropy' ou 'log_loss'
    max_depth=5,               # Profundidade máxima (controla overfitting)
    min_samples_split=10,      # Mínimo de amostras para dividir um nó
    min_samples_leaf=5,        # Mínimo de amostras em um nó folha
    max_features=None,         # Número de atributos a considerar por divisão
    ccp_alpha=0.01,            # Poda de complexidade de custo (post-pruning)
    class_weight='balanced',   # Ajusta pesos para classes desbalanceadas
    random_state=42
)
dt.fit(X_train, y_train)

# Plotar a árvore
plt.figure(figsize=(12, 6))
plot_tree(dt, feature_names=X.columns, class_names=True, filled=True)
plt.show()
```

3.3. Random Forest (Bagging)

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,         # Quantidade de árvores no ensemble
    criterion='gini',         # 'gini' ou 'entropy'
    max_depth=None,           # Profundidade máxima
    max_features='sqrt',      # Atributos sorteados por nó ('sqrt', 'log2', int ou float)
    bootstrap=True,           # Amostragem com reposição
    oob_score=True,           # Calcula o erro Out-Of-Bag
    class_weight='balanced', # Para desbalanceamento
    n_jobs=-1,                # Usa todos os núcleos da CPU
    random_state=42
)
rf.fit(X_train, y_train)

# Importância dos atributos
importancias = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
print("Importância das Variáveis:\n", importancias)
```

3.4. AdaBoost (Boosting)

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

# Estimador base fraco (stump = profundidade 1 por padrão)
base_est = DecisionTreeClassifier(max_depth=1)

ada = AdaBoostClassifier(
    estimator=base_est,       # Modelo base
    n_estimators=50,          # Número de rodadas de boosting
    learning_rate=1.0,        # Taxa de aprendizado (peso de cada modelo)
    random_state=42
)
ada.fit(X_train, y_train)
```

3.5. Naive Bayes

Dica: Não requer normalização de escala; muito rápido e eficiente para texto.

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB

# GaussianNB: para atributos contínuos (assume distribuição normal)
```

```
gnb = GaussianNB()

# MultinomialNB: para contagens de termos/frequências (NLP/Texto)
mnb = MultinomialNB(alpha=1.0)  # alpha = suavização de Laplace

# BernoulliNB: para atributos binários (0 ou 1)
bnb = BernoulliNB(alpha=1.0)
```

3.6. Support Vector Machines (SVM)

Dica: Sempre normalize os dados antes de treinar SVM!

```
from sklearn.svm import SVC

svm = SVC(
    C=1.0,                  # Penalidade do erro (C alto = menos tolerante a erros, risco de overfit)
    kernel='rbf',           # 'linear', 'poly', 'rbf', 'sigmoid'
    gamma='scale',          # 'scale' ou 'auto' (influência do raio no RBF)
    degree=3,               # Grau do polinômio quando kernel='poly'
    class_weight='balanced', # Ajusta margem para classes desbalanceadas
    probability=True,       # Habilita predict_proba (necessário para curva ROC)
    random_state=42
)
svm.fit(X_train_scaled, y_train)
```

---

4. Avaliação de Modelos e Métricas

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    roc_auc_score, RocCurveDisplay
)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba") else None

# Relatório Completo
print("Acurácia :", accuracy_score(y_test, y_pred))
print("Precisão :", precision_score(y_test, y_pred, average='weighted')) # ou 'macro', 'binary'
print("Recall   :", recall_score(y_test, y_pred, average='weighted'))
print("F1-Score :", f1_score(y_test, y_pred, average='weighted'))

print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))

# Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.classes_)
disp.plot(cmap='Blues')
plt.title("Matriz de Confusão")
plt.show()

# Curva ROC e AUC
if y_prob is not None:
    print("ROC-AUC Score:", roc_auc_score(y_test, y_prob))
    RocCurveDisplay.from_predictions(y_test, y_prob)
    plt.show()
```

Guia Rápido de Interpretação de Métricas:

- **Acurácia:** Acertos totais / Total de amostras (cuidado: enganosa em bases desbalanceadas).
- **Precisão:** De todos que o modelo disse que são positivos, quantos realmente são? (Evita falso positivo).
- **Recall (Sensibilidade):** De todos os positivos reais, quantos o modelo encontrou? (Evita falso negativo, vital em medicina/fraude).
- **F1-Score:** Média harmônica entre Precisão e Recall.
- **Opções do parâmetro average em multisseleção:**
  - `'binary'`: para classificação binária (default).
  - `'macro'`: média simples das métricas de cada classe (todas as classes pesam igual).
  - `'weighted'`: média ponderada pelo número de exemplos de cada classe (suporte).

---

5. Validação Cruzada e Otimização de Hiperparâmetros

Validação Cruzada Estratificada

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring='f1_macro')
print(f"F1 Médio: {scores.mean():.4f} (+/- {scores.std():.4f})")
```

Busca em Grade (GridSearchCV)

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'max_features': ['sqrt', 'log2']
}

grid = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)
grid.fit(X_train, y_train)

print("Melhores Parâmetros:", grid.best_params_)
print("Melhor Score:", grid.best_score_)
best_model = grid.best_estimator_
```

Busca Aleatória (RandomizedSearchCV)

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_dist = {
    'n_estimators': randint(50, 300),
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': randint(2, 11)
}

random_search = RandomizedSearchCV(
    estimator=RandomForestClassifier(random_state=42),
```

```
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring='f1_macro',
    random_state=42,
    n_jobs=-1
)
random_search.fit(X_train, y_train)
```

---

6. Distâncias e Similaridades

```
from sklearn.metrics import pairwise_distances
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import jaccard_score

# Distâncias entre matrizes (linhas = amostras)
# Métricas: 'euclidean', 'manhattan', 'minkowski', 'cosine', 'jaccard'
dist_eucl = pairwise_distances(X, metric='euclidean')
dist_manh = pairwise_distances(X, metric='manhattan')

# Similaridade de Cosseno (para texto e vetores de termos)
sim_cos = cosine_similarity(X)

# Coeficiente de Jaccard (para atributos binários assimétricos)
# Ex: vetores binários y_true e y_pred
jacc = jaccard_score([1, 0, 1, 1], [1, 1, 0, 1])
```